
多変量解析及び大規模計算論

第11回：大規模計算論パート

演習：並列処理（確率分布の実現値近似）

講師：玉森，内種

目次

- 振り返り
 - 並列コンピューティング
 - ベイズの定理
- 確率分布の粒子近似
 - マージソート
- 並列化と処理速度比較
 - 演習課題

振り返り

- 並列コンピューティング
 - コンパイラレベル(SIMD, パイプライン)
 - 型にはまると早いが応用はきかない
 - マルチコアレベル(スレッド, OpenMP)
 - 応用範囲が広い
 - 計算クラスタレベル(MPI, HADOOP)
 - 環境構築が大変

振り返り

- 並列コンピューティング
 - コンパイラレベル(SIMD, パイプライン)
 - 型にはまると早いが応用はきかない
 - マルチコアレベル(スレッド, OpenMP)
 - 応用範囲が広い
 - 計算クラスタレベル(MPI, HADOOP)
 - 環境構築が大変

第11/12回への接続

- ベイズの定理と計算困難性
 - $p(\theta \mid D) = \frac{p(D \mid \theta)p(\theta)}{p(D)}$
 - 周辺化積分が計算不能 (θ はパラメータベクトル)
 - $p(D) = \int p(D \mid \theta)p(\theta)d\theta$
 - 高次元では指数的爆発
- 分布の実現値近似 (Monte Carlo近似)
 - $E[f(\theta)] \approx \frac{1}{N} \sum_{i=1}^N f(\theta^{(i)})$

第11/12回への接続

- 確率分布を実現値で近似する意味
 - 確率分布から独立してサンプルしたものが実現値
 - 独立しているので個別に計算可能（並列計算）
- 確率分布を計算機で扱えるとできること
 - 粒子フィルタ（時系列予測，自己位置推定）
 - 分布による探索（ベイズ最適化，進化計算）

マージソート

プログラムでスレッド並列処理する方法
ベイズの定理は出てきません

ソートアルゴリズム

- クイックソート
 - 理論上最速（並列化すると遅くなるかも）
- バブルソート
 - アルゴリズムの勉強用（実務で使うと首飛ぶやつ）
- マージソート
 - 並列化可能！（今回の主役）

クイックソート

1.基準値の選択:

1. [38, 27, 43, 3, 9, 82, 10]最初の要素 38 を基準値に

2.分割:

1. 基準値 38 より小さい部分リスト: [27, 3, 9, 10]
2. 基準値と等しい部分リスト: [38]
3. 基準値 38 より大きい部分リスト: [43, 82]

1.再帰的ソート:

1. [27, 3, 9, 10] に対して同じ手順を適用:

1. 基準値 27 を選択

2. 分割:

1. [3, 9, 10] (27 より小さい部分)

2. [27]

3. [] (27 より大きい部分)

3. 再帰的ソート:

1. 基準値の選択→分割を1要素になるまで繰り返し

1.結合:

1. 小さい部分リスト [3, 9, 10]、基準値 [27]、大きい部分リスト [] を結合して [3, 9, 10, 27]
2. 上記のリスト [3, 9, 10, 27]、基準値 [38]、大きい部分リスト [43, 82] を結合して最終結果 [3, 9, 10, 27, 38, 43, 82]

• 最終結果

- ソートされたリスト: [3, 9, 10, 27, 38, 43, 82]

マージソート

「分割統治法」に基づくソートアルゴリズム

1.分割 (Divide) :

1. リストの要素が1つになるまでリストを半分分割
2. この操作を再帰的に繰り返し小さなリストに分割

2.統治 (Conquer) :

1. 分割された小さなリストをそれぞれソートします。

3.統合 (Combine) :

1. ソートされた小さなリストを順序を保ちながらマージして、大きなソート済みリストを作成

マージソート具体例

1.分割:

1. 初回分割: $[38, 27, 43, 3, 9, 82, 10] \rightarrow [38, 27, 43]$ と $[3, 9, 82, 10]$

2. さらに分割:

1. $[38, 27, 43] \rightarrow [38]$ と $[27, 43]$

2. $[27, 43] \rightarrow [27]$ と $[43]$

3. $[3, 9, 82, 10] \rightarrow [3, 9]$ と $[82, 10]$

4. $[3, 9] \rightarrow [3]$ と $[9]$

5. $[82, 10] \rightarrow [82]$ と $[10]$

マージソート具体例

ソート（最小単位のリストの比較）：

[38], [27], [43], [3], [9], [82], [10] はソート済み

並列化のポイント：

- 1要素まで分割しなくてもコア数まで分割すれば十分
- 例：8コアCPUで100要素だと12,13,12,13,12,13,12,13に分割すればOK
 - 各CPUが受け持つデータ数が均一（負荷が均一）
 - ↑並列化効率が良い
- クイックソートも並列化できるがCPUごとに分割されるデータ数が均一でない（負荷が不均一）

マージソート具体例

1. マージ:

1. [27] と [43] をマージして [27, 43]
2. [3] と [9] をマージして [3, 9]
3. [82] と [10] をマージして [10, 82]

2. さらにマージ:

1. [38] と [27, 43] をマージして [27, 38, 43]
2. [3, 9] と [10, 82] をマージして [3, 9, 10, 82]

マージソート具体例

1.最後にマージ:

1. [27, 38, 43] と [3, 9, 10, 82] をマージして最終結果 [3, 9, 10, 27, 38, 43, 82]

- **最終結果**

- ソートされたリスト: [3, 9, 10, 27, 38, 43, 82]
- 一連の動きをプログラムで見てください
 - Moodleから`merge\_sort\_single.ipynb`を確認

マージソート演習

マージソート演習

- 並列数とデータ数を変えて実行時間を計測
- 結果をグラフ化して考察レポートを提出
 - グラフ（データ数は10,000,000で固定）
 - 横軸：並列数，縦軸：実行時間
 - 考察
 - 並列数を増やすと反比例して実行時間が減るか？

マージソート演習

- プログラム実行方法
 - Moodleから`exercise\_11\_1\_merge\_sort.ipynb`を取得
 - Google colabでWorker数[1,2,3,4,5,6,7,8]で複数回実行
 - 結果を集計（横軸：Worker数，縦軸：実行時間）
 - 反比例していると見てよいか考察
 - 原因を考察

マージソート演習

- Google colabでは2CPUしか動かない



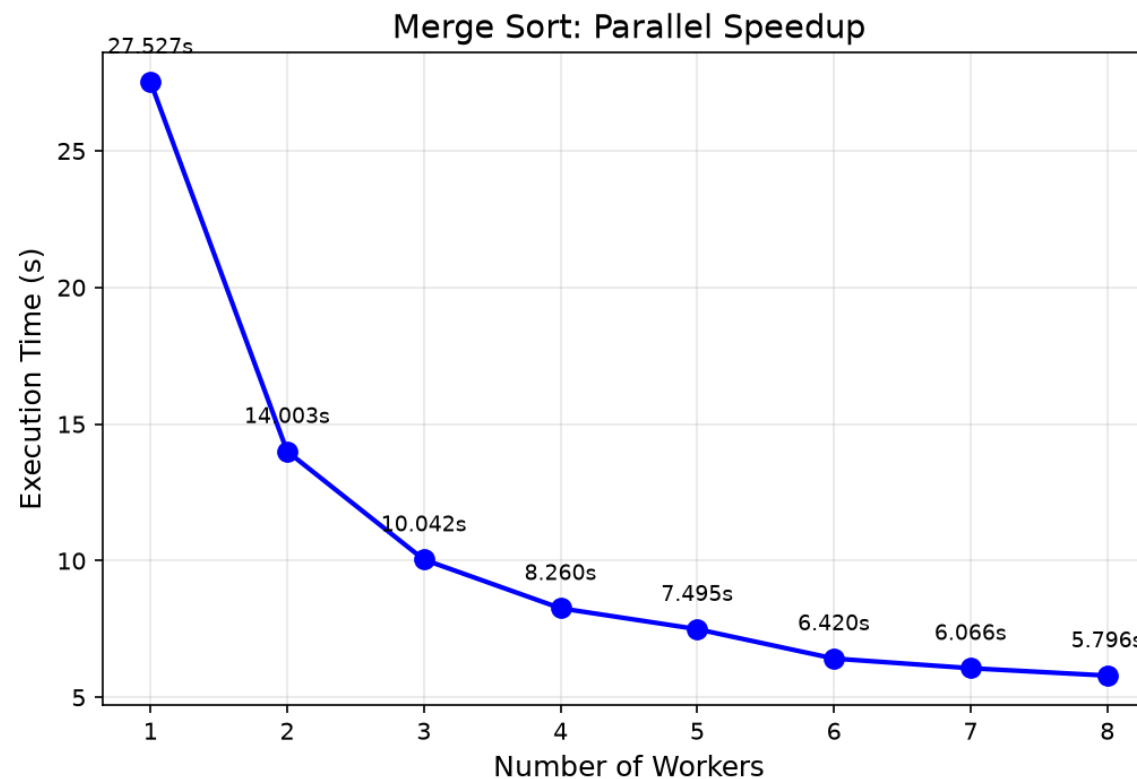
- 手元のノートPCで実行しても良い
- 配布データ`Mac\_mini\_m1\_result.csv`を利用しても良い

Mac_mini_m1_result.csv

num_thred1	num_thred2	num_thred3	num_thred4	num_thred5	num_thred6	num_thred7	num_thred8
24.4705381	13.3751318	10.2021937	8.75088525	8.99721909	8.66719699	8.78241014	8.64183688
24.160676	13.2901282	10.5113664	8.80018997	9.75223017	9.06830788	8.77627301	8.57812619
25.408757	13.5731809	10.3014228	8.81487203	8.89821196	9.41261077	8.76532817	8.57942605
24.677716	13.4912679	10.535233	10.0024118	9.5637238	9.47180414	8.90429807	8.98187327
25.5099549	14.0770681	10.448221	9.38299227	9.54216075	8.25838518	8.30588913	8.41812396

グラフ化の例

- コア数と実行時間の関係
- 反比例の線($y=a/x$)と比較し考察



粒子フィルタ

一般状態空間モデル

樋口知之, 【解説】粒子フィルタ,
電子情報通信学会誌, vol 88, No 12, 2005

今時刻 t の m 次元ベクトルの観測時系列を $\mathbf{y}_t (t=1, \dots, T)$ とする. この観測値に関連した, 観測値を表現するのに必要な変量を一つのベクトルに納めたものを $\mathbf{x}_t$ と書くことにする. この $\mathbf{x}_t$ は状態ベクトルと呼ばれ, 観測することはできない量 (統計では潜在変数という) である. 状態空間モデルと通常呼ばれる, 線形・ガウス型の状態空間モデルは, これらの確率変数間の関係を, 以下のような線形モデルと正規ノイズの二つの式で表したものである.

$$\mathbf{x}_t = \mathbf{F}_t \mathbf{x}_{t-1} + \mathbf{G}_t \mathbf{v}_t \quad [\text{システムモデル}] \quad (1)$$

$$\mathbf{y}_t = \mathbf{H}_t \mathbf{x}_t + \mathbf{w}_t \quad [\text{観測モデル}] \quad (2)$$

ただし, $\mathbf{x}_t$ は k 次元の状態ベクトル, $\mathbf{v}_t \sim N(0, \Sigma_{1t})$ と $\mathbf{w}_t \sim N(0, \Sigma_{2t})$ はそれぞれ l 次元及び m 次元の正規白色雑音で, システムノイズ, 観測ノイズと呼ばれる. Σ_{1t} , Σ_{2t} は分散共分散行列である. また, $\mathbf{F}_t$, $\mathbf{G}_t$, $\mathbf{H}_t$ はそれぞれ $k \times k$, $k \times l$, 及び $m \times k$ 次元の行列である.

非線形・非ガウス型状態空間モデルを更に一般化したものとして, 条件付分布を用いて表現された以下のような一般状態空間モデルを考えることができる.

$$\mathbf{x}_t \sim Q_t(\cdot | \mathbf{x}_{t-1}) \quad [\text{システムモデル}] \quad (5)$$

$$\mathbf{y}_t \sim R_t(\cdot | \mathbf{x}_t) \quad [\text{観測モデル}] \quad (6)$$

ただし, Q_t と R_t はそれぞれ状態 $\mathbf{x}_{t-1}$ と $\mathbf{x}_t$ が与えられたときの条件付分布を表す. このモデルにより, 観測値 $\mathbf{y}_t$ が 2 項分布やポアソン分布に従う count data のモデル化なども可能になる. 音声情報処理で利用されている隠れマルコフモデルは $\mathbf{x}_t$ も離散値のみ許される一般状態空間モデルの特殊なケースである. 一般状態空間モデルで記述される確率変数間の関係をグラフ表現すると, 図 2 のように鎖状のグラフィカルモデルとなる⁽⁵⁾. 状態の時間推移を記述するシステムモデルは図の右矢印に, 状態と観測値の関係を表現する観測モデルは上矢印に対応する.

一般状態空間モデル

樋口知之, 【解説】粒子フィルタ,
電子情報通信学会誌, vol 88, No 12, 2005

鎖状構造グラフィカルモデル

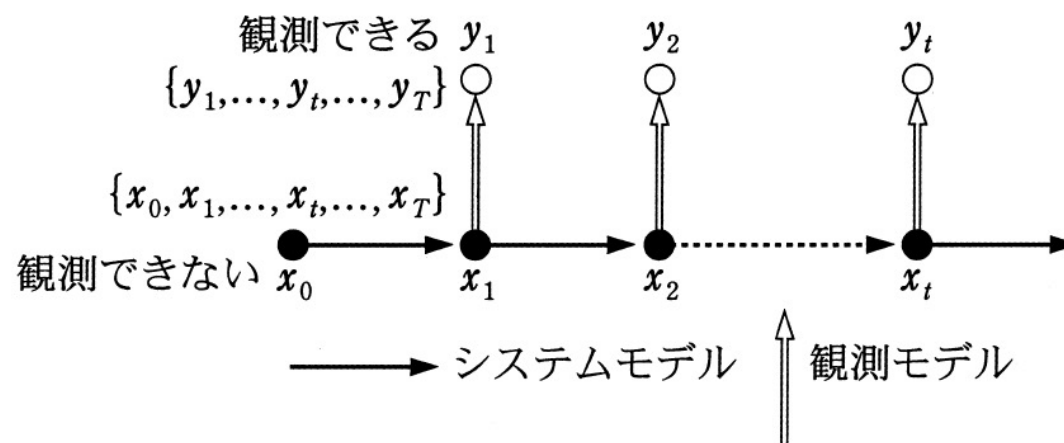


図2 一般状態空間モデルのグラフィカルモデル表現 右向き矢印の状態の時間推移を記述する部分がシステムモデルを, また上向き矢印の状態と観測値の関係を表現する部分が観測モデルを表す. 隠れマルコフモデルは一般状態空間モデルの特殊ケースである. また動的計画法が適用できる最適化問題もこのモデルの状態推定問題として表現できる.

予測とフィルタ

樋口知之, 【解説】 粒子フィルタ,
電子情報通信学会誌, vol 88, No 12, 2005

[一期先予測 (prediction)]

$$p(\mathbf{x}_t | \mathbf{y}_{1:t-1}) =$$

$$\int p(\mathbf{x}_t | \mathbf{x}_{t-1}) p(\mathbf{x}_{t-1} | \mathbf{y}_{1:t-1}) d\mathbf{x}_{t-1} \quad (7)$$

時刻 $t-1$ の分布を積分できればよい
= 時刻 $t-1$ の実現値を時刻 t の実現値に
更新できればよい

[フィルタ (filtering)]

$$p(\mathbf{x}_t | \mathbf{y}_{1:t}) = \frac{p(\mathbf{y}_t | \mathbf{x}_t) p(\mathbf{x}_t | \mathbf{y}_{1:t-1})}{p(\mathbf{y}_t | \mathbf{y}_{1:t-1})} \quad (8)$$

$$\text{ただし, } p(\mathbf{y}_t | \mathbf{y}_{1:t-1}) = \int p(\mathbf{y}_t | \mathbf{x}_t) p(\mathbf{x}_t | \mathbf{y}_{1:t-1}) d\mathbf{x}_t$$

である.

予測とフィルタ

樋口知之, 【解説】 粒子フィルタ,
電子情報通信学会誌, vol 88, No 12, 2005

[一期先予測 (prediction)]

$$p(\mathbf{x}_t | \mathbf{y}_{1:t-1}) =$$

$$\int p(\mathbf{x}_t | \mathbf{x}_{t-1}) p(\mathbf{x}_{t-1} | \mathbf{y}_{1:t-1}) d\mathbf{x}_{t-1} \quad (7)$$

時刻 $t-1$ の分布を積分できればよい
= 時刻 $t-1$ の実現値を時刻 t の実現値に
更新できればよい

[フィルタ (filtering)]

$$p(\mathbf{x}_t | \mathbf{y}_{1:t}) = \frac{p(\mathbf{y}_t | \mathbf{x}_t) p(\mathbf{x}_t | \mathbf{y}_{1:t-1})}{p(\mathbf{y}_t | \mathbf{y}_{1:t-1})} \quad (8)$$

時刻 t の実現値に対して
観測値との尤もらしさを計算

$$\text{ただし, } p(\mathbf{y}_t | \mathbf{y}_{1:t-1}) = \int p(\mathbf{y}_t | \mathbf{x}_t) p(\mathbf{x}_t | \mathbf{y}_{1:t-1}) d\mathbf{x}_t$$

である.

予測とフィルタ

樋口知之, 【解説】 粒子フィルタ,
電子情報通信学会誌, vol 88, No 12, 2005

[一期先予測 (prediction)]

$$p(\boxed{x_t} | \boxed{y_{1:t-1}}) =$$

$$\int p(\boxed{x_t} | x_{t-1}) p(x_{t-1} | \boxed{y_{1:t-1}}) dx_{t-1} \quad (7)$$

時刻t-1の分布を積分できればよい
= 時刻t-1の実現値を時刻tの実現値に
更新できればよい

[フィルタ (filtering)]

$$p(\boxed{x_t} | \boxed{y_{1:t}}) = \frac{p(y_t | x_t) p(\boxed{x_t} | \boxed{y_{1:t-1}})}{p(y_t | y_{1:t-1})} \quad (8)$$

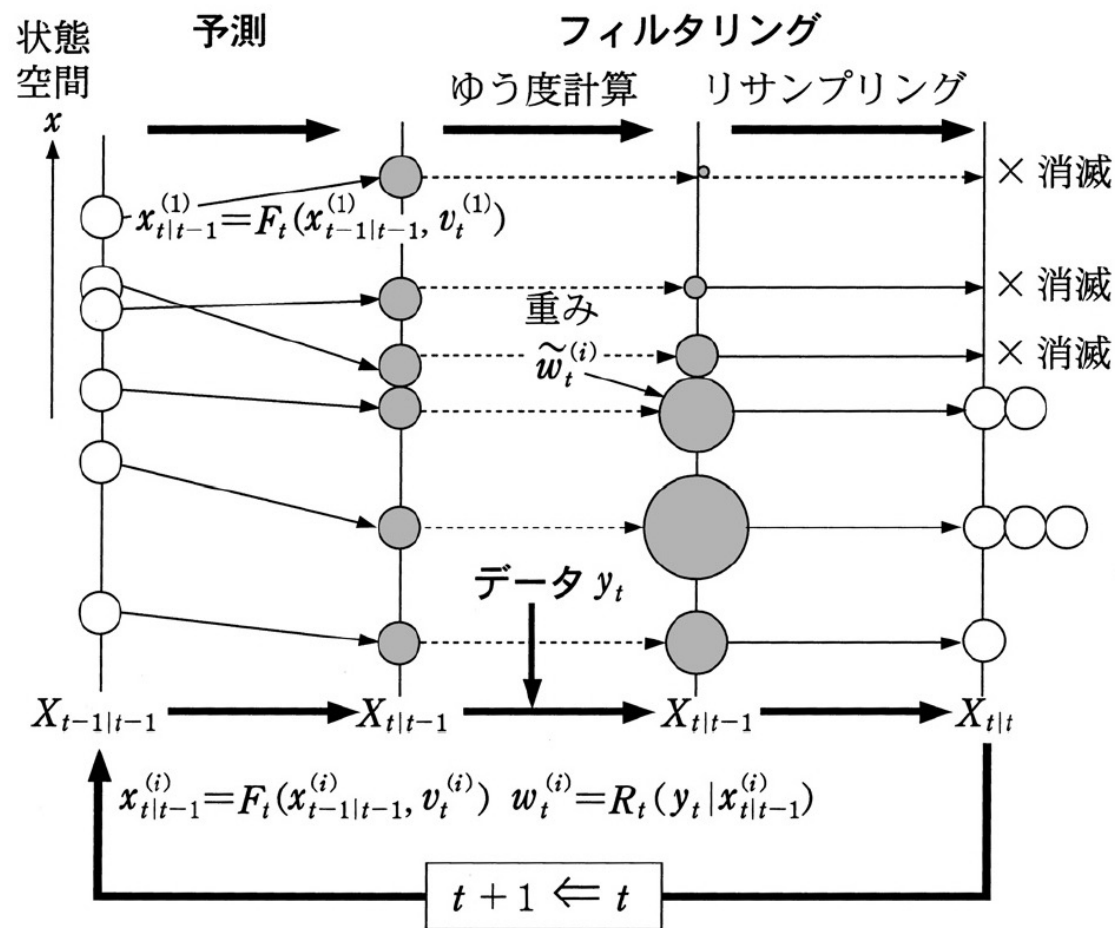
時刻tの実現値に対して
観測値との尤もらしさを計算

$$\text{ただし, } p(y_t | y_{1:t-1}) = \int p(y_t | x_t) p(x_t | y_{1:t-1}) dx_t$$

である.

アルゴリズム

樋口知之, 【解説】粒子フィルタ,
電子情報通信学会誌, vol 88, No 12, 2005



スレッド並列化できるのはどこ？

- 予測：粒子ごと or 全粒子
- 尤度計算：粒子ごと or 全粒子
- リサンプリング：粒子ごと or 全粒子

スレッド並列化できるのはどこ？

- 予測：粒子ごと or 全粒子
- 尤度計算：粒子ごと or 全粒子
- リサンプリング：粒子ごと or 全粒子

- 予測と尤度計算は並列化可能
 - はたして並列化できるが常に有効だろうか？
 - 粒子数増やしても並列化すると時短できる？
 - リサンプリング処理をスキップできれば高速化？

粒子フィルタ演習

粒子フィルタ演習

- プログラム実行方法

- Moodleから`exercise\_11\_2\_particle\_filter.ipynb`を取得
- 並列処理の箇所を確認

```
with multiprocessing.Pool(processes=num_workers) as pool:
```

- Google colabで実行

- 粒子数[10,50,200,800]
- Woker数[1,2,4,8]
- 集計（横軸：Woker数，縦軸：精度・実行時間）
- 精度と実行時間のトレードオフを考察

粒子フィルタ演習

- システムモデルと観測モデル

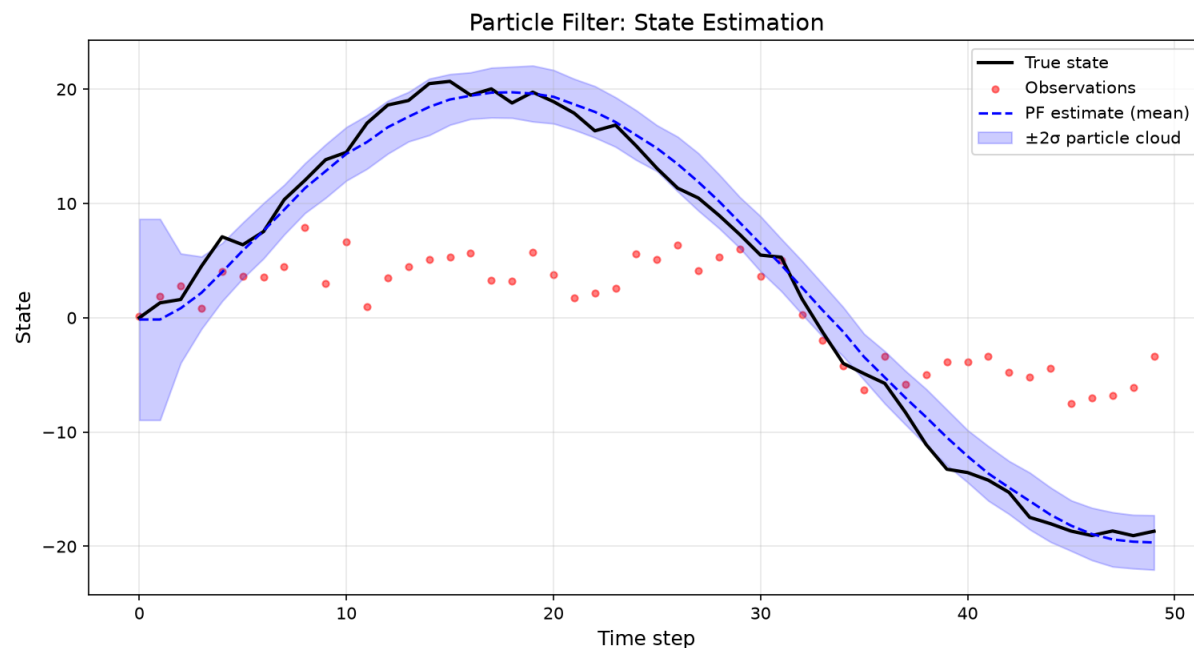
$$x_t = 0.5 x_{t-1} + 10 \sin(0.1 t) + w_t, w_t \sim N(0, Q)$$

$$y_t = x_t / (1 + 0.01 x_t^2) + v_t, v_t \sim N(0, R)$$

- 結果をグラフ化して考察レポートを提出
 - 横軸：Worker数, 縦軸：精度（真の状態 x とのRMSE）
 - 横軸：並列数, 縦軸：実行時間

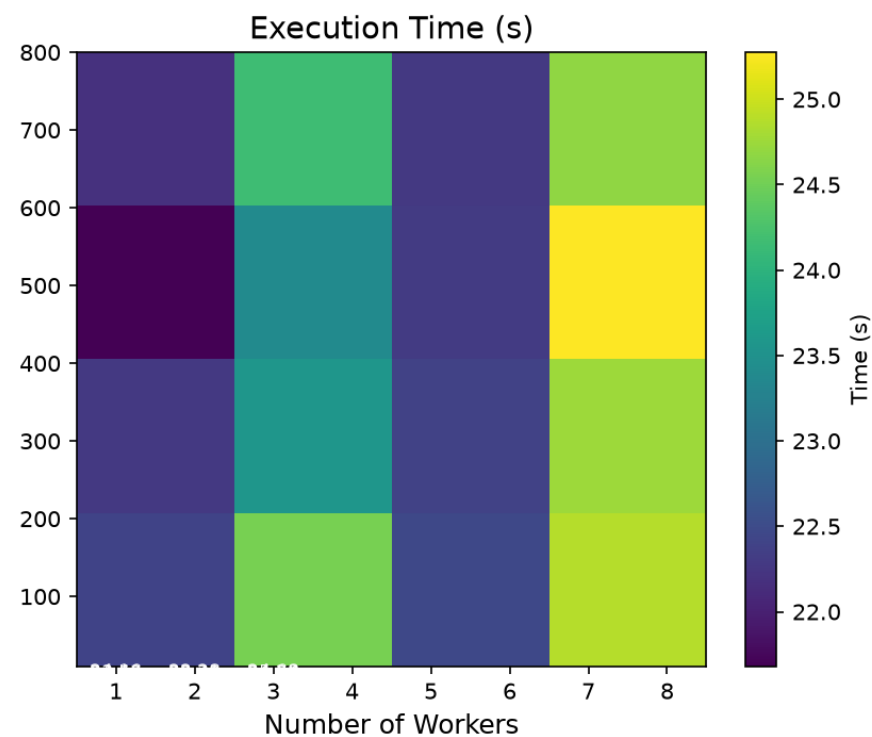
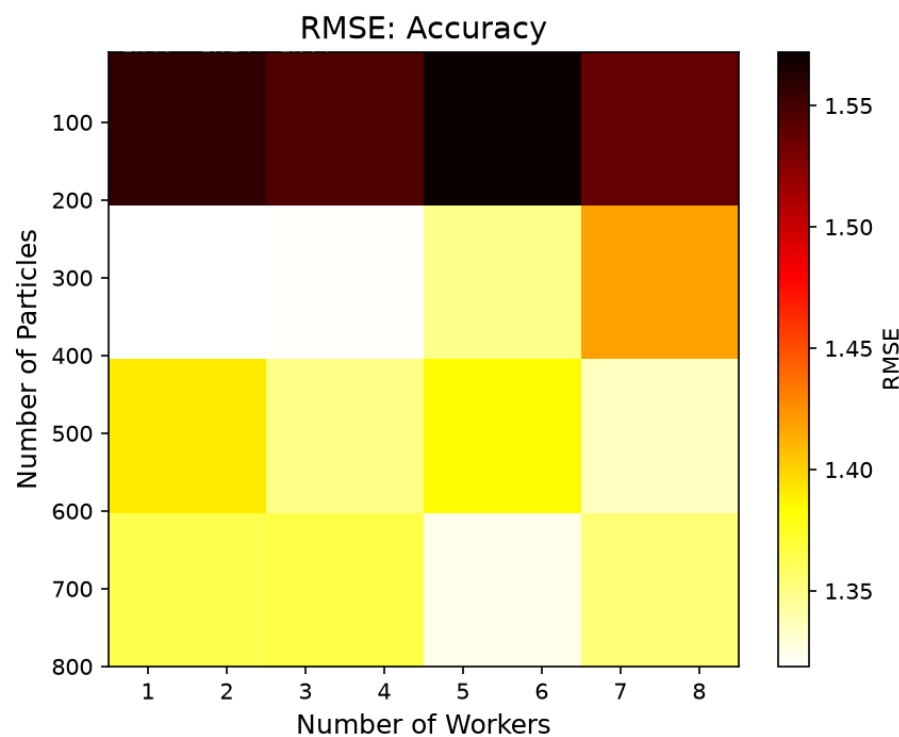
粒子フィルタ

- Google colabでは2CPUしか動かない
- 精度は変わらないが実行時間は手元のノートPCで実行しても良い



グラフ化の例

- Woker数と粒子数と精度の関係
- Woker数と粒子数と実行時間の関係



第11回まとめ

- ソートの並列化
 - クイックソート：並列化不可
 - マージソート：並列化可能
- 粒子フィルタ
 - 粒子毎に予測計算・重み計算が並列化可能
- 課題: 並列化効果について考察
 - Woker数と処理時間の関係？
 - 粒子フィルタの処理時間と精度の関係？